

Penguide: An Educational Autonomous Agent for Linux and Kernel Concepts Using Local Large Language Models

Authors: [Your Name], [Advisor/Co-authors]

Affiliation: [Your Institution]

Contact: [Your Email]

Abstract

Learning the Linux command line and operating-system concepts is essential for computing education but often daunting for beginners. We present **Penguide**, an educational autonomous agent that assists users in learning Linux CLI operations and underlying kernel concepts through natural-language dialogue. Penguide combines a local large language model (LLM) with a curated kernel-documentation knowledge base, multi-turn conversation memory, and a permit-list sandbox for safe command execution. The system follows a *teaching-first* design: it explains each command or concept before executing any action and grounds explanations in kernel documentation when relevant. We describe the architecture, the end-to-end data flow, and the safety and configurability measures. The implementation is lightweight (Python, Ollama), runs entirely on-premises for privacy, and is extensible with optional training pipelines. We discuss design rationale, automated validation, and directions for evaluation and future work.

Keywords: educational AI, Linux systems education, conversational agent, large language models, knowledge grounding, command-line interface, kernel documentation, safe execution.

1. Introduction

1.1 Motivation

Proficiency in the Linux command line and an understanding of operating-system principles are core learning outcomes in computer science and systems courses [1,2]. Students and self-learners, however, often struggle with the breadth of commands, the relationship between user-space tools and kernel behavior, and the risk of running destructive commands. Traditional resources—man pages, wikis, and static tutorials—require learners to map their intent to the right command and to connect CLI behavior to kernel concepts. Interactive tutoring systems that explain *why* a command is suggested and that link commands to kernel subsystems can improve both engagement and conceptual understanding [3,4].

1.2 Problem and Goals

We address the design and implementation of an **educational autonomous agent** that:

1. Accepts natural-language questions about Linux and the kernel.
2. Explains commands and concepts in simple terms before performing any action.
3. Suggests at most one safe shell command per turn and can execute it in a controlled environment.
4. Grounds explanations in curated kernel-documentation snippets when relevant.
5. Maintains multi-turn conversation context so users can ask follow-up questions.
6. Operates locally (on-premises) to preserve privacy and to avoid dependence on cloud APIs.

1.3 Contributions

We make the following contributions:

- **System design:** A modular architecture that integrates an LLM, a keyword-based knowledge base over kernel documentation, conversation memory with a bounded context window, and a permit-list shell sandbox, orchestrated by a single step function that handles both explanatory and actionable turns.
- **Implementation:** A working prototype, Penguide, implemented in Python using the Ollama API for local LLM inference, with configurable settings and explicit error handling.
- **Safety and configurability:** A permit-list and deny-list policy for command execution, configurable via code and environment variables, documented for deployers and educators.
- **Documentation and extensibility:** End-to-end documentation of data flow and components, plus optional scripts for fetching training data and training a small TensorFlow model, intended for future use on separate hardware.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 presents the system design and methodology. Section 4 summarizes the implementation. Section 5 discusses validation and limitations. Section 6 concludes and outlines future work.

2. Related Work

Educational agents and intelligent tutoring systems. ITS and conversational agents have a long history in education [5,6]. Recent work has focused

on integrating large language models for open-ended dialogue and explanation generation [7,8]. Penguide fits in this line by using an LLM as the primary generator of explanations and command suggestions while constraining behavior through prompts and tool use.

CLI and shell assistance. Prior systems have addressed command prediction, natural-language-to-command translation, and shell scripting assistance [9,10]. Some rely on cloud APIs or proprietary models; others use local models or rule-based components. Penguide emphasizes *education* (explanation before execution) and *safety* (permit-list execution and one command per turn), rather than raw automation.

Knowledge grounding and RAG. Retrieval-augmented generation (RAG) is widely used to ground LLM outputs in external knowledge [11]. Penguide uses a lightweight form of grounding: a curated set of kernel-documentation snippets is loaded at startup, and a simple keyword/topic matcher selects relevant snippets to inject into the prompt. This avoids embedding and vector search in the current prototype while still providing kernel context; extending to semantic retrieval is a natural next step.

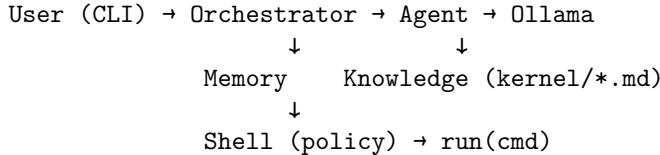
Local and open-source LLMs. Tools such as Ollama [12] and llama.cpp enable local inference of open-weight models, reducing latency and preserving privacy. Penguide is built on Ollama to keep all user input and dialogue on the user’s machine, which is important in educational and sensitive environments.

3. System Design and Methodology

3.1 High-Level Architecture

Penguide is organized around a central **orchestrator** that, for each user turn, (1) retrieves relevant kernel documentation, (2) gathers recent conversation history, (3) constructs a prompt for the LLM, (4) parses the model reply for an optional command, (5) executes the command in a sandbox if present, and (6) updates memory and returns the response. Figure 1 gives a component-level view.

Figure 1 — Component architecture. User input enters via the CLI; the orchestrator coordinates memory, knowledge base, and agent. The agent calls the Ollama API; if the reply contains RUN: <cmd>, the shell tool runs the command subject to the allow/deny policy.



Components:

- **Entry (CLI):** A single entry point supports an interactive loop (user types at a > prompt) or a one-shot mode (query passed as command-line arguments). A “clear” action resets conversation memory.
- **Orchestrator:** Owns the agent (LLM client), memory, and knowledge base; implements the step function that runs the pipeline for one user message.
- **Agent:** Wraps the Ollama API: it builds the full prompt (system prompt + user message) and prepends context (history + knowledge snippets), then returns the raw model reply.
- **Memory:** Stores a list of (role, content) entries (user, assistant, tool). When building context, only the last N turns are included to avoid exceeding the model’s context window.
- **Knowledge base:** Loads all Markdown files from a configurable path (e.g. `knowledge/kernel/`). On each query, it performs keyword/topic matching against filenames and content and returns concatenated snippets to inject into the prompt.
- **Shell tool:** Executes a command only if its first token is in an allow list and not in a deny list; runs with a fixed timeout and returns stdout/stderr or an error message.
- **Config:** Central settings (Ollama URL, model name, timeouts, memory turn limit, knowledge path) are read from environment variables with safe defaults; the allow/deny lists are defined in a policy module.

This design keeps the LLM as the single generative component while clearly separating retrieval (knowledge, memory), safety (policy, shell), and configuration.

3.2 End-to-End Data Flow (One Turn)

For a single user message, the pipeline proceeds as follows:

1. **Retrieval:** The orchestrator calls the knowledge base with the user input; the KB returns a string of relevant kernel-doc snippets (or empty). The orchestrator then obtains the last N turns from memory, formatted as “Role: content” lines.
2. **Prompt construction:** Full context = history string + (optionally) knowledge string. The prompt is the system prompt (defining Penguide’s role and the rule “one command per response, format RUN: <command>”) plus “User: <user_input>”. The agent sends (context, prompt) to the LLM as a single concatenated prompt.
3. **Model reply:** The LLM returns a free-text reply. The orchestrator stores the user message in memory.
4. **Parsing:** A regex searches for RUN: <command>. If no match, the reply is treated as explanation-only; it is stored as the assistant turn and returned to the user.
5. **Execution (if RUN present):** The substring before RUN: is the explanation; the rest is the command. The shell tool checks the command’s first token against the allow/deny lists. If allowed, the command is run in a

subprocess with a timeout; the tool returns stdout or an error message. The orchestrator formats a combined response (explanation + “Executing: `<cmd>`” + output), stores tool output and assistant response in memory, and returns the combined response to the user.

Thus each turn is deterministic in structure: retrieval → prompt → one LLM call → optional one command execution → memory update.

3.3 Teaching-First and Safety Design

Teaching-first: The system prompt instructs the model to always explain the purpose of a command or concept in simple terms. When the user asks for an action, the model must output an explanation followed by `RUN: <command>`. The UI presents the explanation and then the execution result, so the learner sees the rationale before the effect.

One command per turn: Limiting to a single `RUN:` per response simplifies parsing and reduces the risk of long or complex command chains. It also encourages step-by-step learning.

Sandbox: Only commands whose first token appears in the allow list and not in the deny list are executed. The allow list includes common read-only and safe write commands (e.g. `ls`, `cat`, `pwd`, `ps`, `grep`, `mkdir`, `cp`); the deny list includes `sudo`, `chmod`, `dd`, `reboot`, etc. Execution uses a subprocess with a fixed timeout. This design is explicitly documented as a first-token check only; argument parsing and privilege escalation are out of scope for the current prototype, and the system should not be run with elevated privileges.

Privacy: The LLM runs via Ollama on the same machine; no dialogue or commands are sent to the cloud by default.

3.4 Knowledge Base and Extensibility

The knowledge base loads Markdown files from a configurable directory. Each file is treated as one document; the query function matches the user input (lowercased) against the filename-derived topic and against words in the input. If “kernel” appears in the input, all documents are included. The result is a concatenation of “From Kernel Doc: `<filename>`” plus content. This provides a simple, interpretable form of grounding without embeddings; extending to semantic search or vector retrieval is left for future work.

The project also includes scripts to fetch external datasets (e.g. Linux command datasets from Hugging Face) and to export the in-repo knowledge as instruction–output pairs. A separate TensorFlow script trains a small LSTM-based model on these pairs for optional use as a fallback or command-suggestion component; training is intended to be run on separate hardware (e.g. another laptop) and is documented accordingly.

4. Implementation

4.1 Technology Stack

- **Language and runtime:** Python 3; no web framework.
- **LLM:** Ollama [12] (local server); HTTP POST to `/api/generate` with `stream: false`. Model name (e.g. Mistral) and URL are configurable via environment variables.
- **Dependencies:** `requests` for the Ollama client; optional `datasets` and `tensorflow` for the training pipeline.
- **Execution:** Single process; interactive CLI or one-shot invocation.

4.2 Key Design Choices

- **Context ordering:** Context (history + knowledge) is passed before the system prompt and user message so the model sees prior dialogue and kernel snippets first.
- **Error handling:** Timeouts, connection errors, and other Ollama failures return short, user-visible messages instead of empty or silent fallbacks. Shell timeouts and execution errors are similarly surfaced.
- **Memory window:** A configurable maximum number of turns is included in the context to avoid token overflow and to keep responses focused on recent dialogue.
- **Configuration:** All operational parameters (Ollama URL, model, timeouts, memory limit, knowledge path, shell timeout) can be overridden by environment variables, supporting different deployments without code changes.

4.3 Project Structure

The codebase is organized into: `bin/` (CLI entry); `core/` (orchestrator, agent, knowledge, prompt); `configs/` (policy and settings); `memory/` (conversation history); `models/` (Ollama client); `tools/` (shell runner and registry); `knowledge/kernel/` (Markdown docs); `tests/` (flow and comprehensive tests); `docs/` (architecture and training documentation); and `scripts/` (data fetch and TensorFlow training).

5. Discussion and Validation

5.1 Automated Tests

The project includes two test suites. One exercises the knowledge base (e.g. query “process management” returns content from processes-related docs) and the full educational flow (a real or failing LLM call still produces a non-empty response path). The other runs unit-style tests: system prompt branding, knowledge retrieval for specific keywords, memory retention across steps (at least two user

and two assistant entries after two turns), policy enforcement (denied commands return empty output; allowed commands such as `pwd` return non-empty), and with a mocked Ollama response, verification that the orchestrator correctly parses explanation and `RUN:` and formats the “Executing:” output. These tests validate the pipeline and safety behavior in a repeatable way.

5.2 Limitations

- **Knowledge retrieval:** Keyword matching may miss relevant docs or include irrelevant ones; semantic retrieval (embeddings + vector search) would likely improve relevance.
- **Safety:** The sandbox is based only on the first token of the command; arguments are not parsed. Dangerous combinations (e.g. `rm` with broad paths) remain possible if `rm` is allowed. The system is intended for controlled, educational use.
- **Evaluation:** No user study or quantitative evaluation of learning outcomes or satisfaction is reported here; such work is planned as future work.
- **Model dependence:** Response quality and adherence to the “explanation + `RUN:`” format depend on the underlying LLM (e.g. Mistral via Ollama). Prompt engineering and optional fine-tuning or a small auxiliary model (as in the training scripts) could improve consistency.

5.3 Possible Evaluation Directions

Future work could include: (1) a user study with beginners to measure perceived helpfulness and conceptual gain; (2) benchmarking command correctness and explanation quality on a fixed set of queries; (3) comparing keyword-based vs. embedding-based knowledge retrieval on relevance metrics; (4) ablation of memory and knowledge components to quantify their contribution to response quality.

6. Conclusion and Future Work

We presented Penguide, an educational autonomous agent that helps users learn the Linux command line and kernel concepts through natural-language dialogue. The system combines a local LLM (Ollama), a curated kernel-documentation knowledge base, multi-turn memory, and a permit-list shell sandbox under a single orchestrator. The design prioritizes teaching-first behavior (explanation before execution), safety (allow/deny list, timeout), and privacy (local inference). We described the architecture, data flow, implementation, and validation approach, and we outlined limitations and directions for evaluation.

Planned future work includes: integrating semantic retrieval for the knowledge base; tightening the execution sandbox (e.g. argument checks or restricted shells); conducting a user study and quantitative evaluation; and optionally integrating

the TensorFlow-trained model as a fallback or suggestion component. The codebase and documentation are structured to support these extensions and to serve as a basis for further research in educational AI for systems and CLI learning.

References

- [1] M. Goldweber, J. Barr, and C. Clear, “Supporting the pipeline: The role of CS in K-16,” in *Proc. SIGCSE*, 2016.
 - [2] T. A. Wagner and R. R. Ressler, “Teaching operating systems: The importance of projects,” in *Proc. SIGCSE*, 1999.
 - [3] B. P. Woolf, *Building Intelligent Interactive Tutors*, Morgan Kaufmann, 2009.
 - [4] K. VanLehn, “The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,” *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
 - [5] A. Graesser et al., “AutoTutor and family: A review of 17 years of natural language tutoring,” *International Journal of Artificial Intelligence in Education*, vol. 28, pp. 524–581, 2018.
 - [6] R. S. Baker, “Stupid tutoring systems, intelligent humans,” *International Journal of Artificial Intelligence in Education*, vol. 26, no. 2, pp. 600–614, 2016.
 - [7] L. K. Fryer et al., “Chatbot learning partners: Connecting learning experiences, interest and competence,” in *Computers in Human Behavior*, vol. 93, pp. 279–289, 2019.
 - [8] O. Kasneci et al., “ChatGPT for good? On opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, 2023.
 - [9] X. Chen et al., “Latent attention for if-then program synthesis,” in *Proc. NeurIPS*, 2016.
 - [10] R. Shin et al., “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, 2021.
 - [11] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP,” in *Proc. NeurIPS*, 2020.
 - [12] Ollama. <https://ollama.ai/>. Accessed 2024.
-

Appendix A: Example Interaction

User: How do I see running processes?

Penguide: In Linux you can list running processes with the `ps` command. Here we’ll use a simple, readable form:

RUN: `ps aux`

> **Executing:** `ps aux`

USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
...

User: What does the kernel do with memory?

Penguide: [Explanation using injected kernel memory-management snippet: virtual memory, paging, slab allocator, etc. No RUN in this turn.]

This paper was prepared for academic submission. Adjust author names, affiliation, and references as required by the target venue (e.g. IEEE, ACM).